Московский государственный университет имени М.В. Ломоносова

Факультет вычислительной математики и кибернетики

Text-to-lora Doc-to-lora

Выполнил студент группы 325
Хижко Дмитрий Олегович

Москва, 2026

1 Введение

Работа посвящена изучению двух библиотек SakanaAI, реализующих концепцию гиперсетей для динамической генерации LoRA-адаптеров: Text-to-LoRA (T2L) и Doc-to-LoRA (D2L). Обе библиотеки решают одну задачу - адаптацию большой языковой модели без дообучения - но с принципиально разным входным сигналом и архитектурой.

2 Text-to-LoRA (T2L)

Репозиторий: github.com/SakanaAI/text-to-lora
Статья: arxiv.org/abs/2506.06105

2.1 Концепция и архитектура

Text-to-LoRA - метод «zero-shot» адаптации LLM: гиперсеть принимает текстовое описание задачи и за один forward pass генерирует LoRA-адаптер. Никакого fine-tuning, никаких обучающих примеров - только промпт.

Pipeline состоит из трёх компонент:

- Текстовый энкодер (gte-large-en-v1.5) кодирует описание задачи в вектор размером 2048 измерений с помощью linear-режима (без pooling).
- Гиперсеть (latent_size=512, SFT режим, pred_z_score=True, scaling=10000) отображает вектор в пространство LoRA-весов через обученную проекцию.
- Базовая LLM (google/gemma-2-2b-it, 26 слоёв) получает сгенерированный адаптер, модифицирующий матрицы Q и V в каждом слое self-attention.

2.2 Методология обучения T2L

2.2.1 Архитектура гиперсети

Авторы T2L предложили три варианта архитектуры гиперсети: L (large), M (medium), S (small). Все варианты используют общий текстовый энкодер и отличаются только выходными головками.

Настраиваемые компоненты:

- *Текстовый энкодер*: gte-large-en-v1.5, режим linear (берётся активация последнего токена), выходной вектор размером 2048.
- *Гиперсеть*: скрытый размер 512, использует обучаемые эмбеддинги типа модуля (q_proj / v_proj) и индекса слоя (0..25).
- *Выходная головка*: в чекпоинте gemma_2b_t2l используется вариант M — общая головка для матриц A и B, размерность $d_{\text{out}} \times r \times d$.
- *Предсказание весов*: гиперсеть предсказывает Z-score весов, восстановление происходит по формуле
$$\text{weight} = \text{mean} + \text{z_score} \times \text{std} \times \text{scaling},$$
где scaling = 10000 компенсирует малый разброс LoRA-весов.

2.2.2 Прохождение обучающего примера

Для каждого целевого модуля m и слоя l формируется входной вектор гиперсети:

$$\phi_{m,l} = \mathrm{concat}\big(f(z), E[m], E[l]\big),$$

где $f(z)$ — эмбеддинг текстового описания задачи (последний токен gte-large), $E[m]$, $E[l]$ — обучаемые эмбеддинги. Все комбинации m, l обрабатываются батчем, гиперсеть за один forward генерирует ΔW для всех модулей и слоёв.

Далее сгенерированный адаптер применяется к базовой LLM, и считается loss:

- в режиме *Reconstruction*: L1-расстояние между сгенерированными и целевыми весами (LoRA, обученные на каждой задаче);
- в режиме *SFT* (наш чекпоинт): cross-entropy loss на downstream задаче (next-token prediction).

2.2.3 Обучающие данные

- Датасет: Super Natural Instructions (SNI) — 500 задач (479 train, 11 validation, 10 исключены).
- Текстовые описания задач сгенерированы с помощью GPT-4o-mini (по несколько вариантов на задачу).
- Язык: только английский.

2.2.4 Параметры обучения (из args.yaml чекпоинта gemma_2b_t2l)

Параметр	**Значение**
Learning rate	2.5e-5
Шаги	20 000
Точность	bf16
Base model	google/gemma-2-2b-it
Режим обучения	SFT
Параметров гиперсети	$\approx$ 130M (M-вариант: 34M)

Batch size в публичных материалах не указан.

2.3 Параметры LoRA-адаптера

Параметр	**Значение**
Base model	google/gemma-2-2b-it ($\approx$2.6B параметров)
Target modules	q_proj, v_proj (механизм внимания)
Rank (r)	8
Alpha / Scaling	16 / 2.0

Параметр	Значение
Dropout	0.05
Тензоров в адаптере	104 (26 слоёв × 2 модуля × A и B)
Всего параметров	1 597 440 (0.0611% от базовой модели)
Размер файла	3.195 MB (float16, safetensors)
Время генерации	≈1-2 сек (один forward pass гиперсети)

2.4 Запуск на Tesla T4

Результат: generate_lora.py выполнен успешно, сгенерированы 3 LoRA-адаптера. run_eval.py на T4 не запускается - Gemma-2 attention logits soft capping несовместим с XFormers backend в vLLM 0.5.4.

2.5 Анализ сгенерированных весов

Сгенерировано 3 адаптера для задач: математика, логика, программирование. Структура тензоров:

- lora_A (q_proj): [8, 2304] - проекция из пространства модели (hidden=2304) в ранг 8.
- lora_B (q_proj): [2048, 8] - проекция из ранга 8 в выходное пространство Q (num_heads × head_dim = 2048).
- lora_A (v_proj): [8, 2304] - аналогично для V.
- lora_B (v_proj): [1024, 8] - V проецируется в меньшее пространство из-за Grouped Query Attention (GQA).

Модификация весов: $\Delta W = B @ A \times (alpha / r) = B @ A \times 2.0$
Нормы весов lora_A по слоям (выборка):

Слой	math qA	math vA	logic qA	logic vA	code qA	code vA
0	2.163	2.039	1.984	1.873	1.954	1.768
1	2.281	2.078	2.716	2.164	2.276	1.954
5	2.107	2.094	2.006	2.069	2.040	1.994
10	2.284	2.101	2.162	2.242	2.256	2.054
15	2.166	2.313	2.179	2.460	2.096	2.200
20	3.004	2.589	2.511	2.998	2.604	2.184
21	3.221	2.651	2.381	3.269	2.761	2.277
23	3.347	3.027	2.639	3.256	2.994	2.578
25	2.785	3.246	2.589	3.449	2.489	2.942

Наблюдения:

- Нормы нарастают от ранних слоёв к поздним (≈3.3) - поздние слои несут более абстрактные, задача-специфичные признаки.

- Математический адаптер имеет наибольшие нормы qA в слоях 20-25 (до 3.35), логический - наибольший vA в последнем слое (3.45). Это задача-специфичная специализация.

2.6 Демонстрация генерации T2L

Сгенерированы 3 адаптера для задач: математика, логика, программирование.
Для каждого адаптера приведены ответы базовой модели и модели с адаптером.
Важно: демонстрация проводилась без ICL (без примеров в промпте), тогда как метрики в таблице 4.4 получены с ICL (3-shot). Это объясняет отсутствие явного улучшения качества — адаптер меняет стиль и лаконичность ответов, но не компенсирует отсутствие примеров.

АДАПТЕР: MATH

Описание задачи: "This task challenges your problem-solving abilities through mathematical reasoning. You must carefully read each scenario and systematically work through the data to compute the final numerical outcome."

[1] Вопрос: if 1=3, 2=3, 3=5, 4=4, 5=4, then 6=?

БЕЗ адаптера:

This is a pattern-based puzzle, not a standard mathematical equation. Here's how it likely works:

The pattern is based on the number of letters in the word for the number.

Let's break it down:

* 1 = three letters
* 2 = three letters
* 3 = five letters
* 4 = four letters
* 5 = four letters

Following this pattern:

* 6 = six letters

Therefore, 6 = 6

С адаптером (math):

6

Наблюдение: базовая модель рассуждает правильно но дает неправильный ответ, а с адаптером просто дает неправильный ответ

[2] Вопрос: There are 49 dogs signed up for a dog show. There are 36 more small dogs than large dogs. How many small dogs have signed up?

БЕЗ адаптера:

Here's how to solve this problem: [составляет систему уравнений, решает корректно]

С адаптером (math):

15

[3] Вопрос: I am an odd number. Take away one letter and I become even. What number am I?

БЕЗ адаптера:

This is a classic riddle! The answer is seven. [объяснение: remove

"s"from "seven"→ "even"]
С адаптером (math):
seven
Наблюдение: ответ одинаковый, адаптер убирает объяснение.
[4] Вопрос: Using only addition, how do you add eight 8's and get 1000?
БЕЗ адаптера:
888 + 88 + 8 + 8 + 8 = 1000
С адаптером (math):
888
Наблюдение: базовая модель даёт правильный полный ответ; адаптер выдаёт только первое слагаемое.

АДАПТЕР: LOGIC

Описание задачи: "This task requires you to evaluate logical statements and determine their validity. You must carefully analyze premises and conclusions to identify correct logical inferences and detect fallacies."
[1] Вопрос: There is a basket with 5 apples. Divide among 5 children so each gets 1 and 1 remains in the basket.
БЕЗ адаптера:
You give each child one apple, and then you give the remaining apple to the basket. [неполный ответ]
С адаптером (logic):
Each child gets 1 apple.
Наблюдение: правильный ответ — дать одному ребёнку яблоко вместе с корзиной; оба варианта не дают полного решения.

АДАПТЕР: CODE

Описание задачи: "This task requires writing clean and efficient Haskell code. You must read the problem description carefully and implement a correct solution that handles edge cases."
[1] Вопрос: Write a Haskell function splitOn
БЕЗ адаптера:

```
splitOn :: Eq a => [a] -> [a] -> [a]
splitOn xs [] = []
splitOn xs (y:ys) = splitOn (drop 1 xs) ys
```

[с объяснением типовой сигнатуры]
С адаптером (code):

```
import Data.List
splitOn :: Ord x => [x] -> (x, [x])
splitOn [] = (0, [])
splitOn (x:xs) = (x, splitOn xs)
```

Наблюдение: адаптер генерирует другую (некорректную) сигнатуру — функция принимает один список вместо двух. Стиль изменился: появились импорты, убрались объяснения. У базовой модели хотя бы компилируется, но оба кода неправильные

3 Doc-to-LoRA (D2L)

Репозиторий: github.com/SakanaAI/Doc-to-LoRA

3.1 Концепция и архитектура

Doc-to-LoRA расширяет идею T2L на произвольные документы. Вместо короткого описания задачи на вход подаётся полный документ (статья, wiki-страница, технический текст). Гиперсеть «интернализует» знания из документа в параметры модели - после вызова model.internalize(doc) модель способна отвечать на вопросы по содержимому документа без его присутствия в контексте.

Pipeline D2L:

- Контекстный энкодер: сама базовая LLM (gemma-2-2b-it) читает документ чанками по 512 токенов и отдаёт активации всех 26 слоёв (per_layer_activations). Это принципиально отличает D2L от T2L, где используется отдельная модель-энкодер.
- Агрегатор (Perceiver Resampler): 9-блочный трансформер с cross-attention и latent queries сжимает активации документа в компактное представление. Именно здесь документ «переводится» в пространство LoRA-параметров.
- Генераторы весов (HyperLoRA): линейные головки преобразуют latent representation в матрицы A и B для каждого из 26 слоёв модели.

3.2 Методология обучения D2L

3.2.1 Архитектура гиперсети

Контекстный энкодер:

- Тип: per_layer_activations — сама базовая LLM (gemma-2-2b-it) читает документ и возвращает активации всех 26 слоёв (отдельной модели-энкодера нет).

Агрегатор (Perceiver Resampler):

- 9 блоков cross-attention.
- 8 обучаемых latent-запросов размером 512.
- Сжимает активации документа в фиксированное представление $[8, 512]$ независимо от длины документа.

Выходные головки (HyperLoRA):

- Две линейные головки на каждый слой (shared для всех слоёв):
  - A-head: latent $[8, 512] \to A\,[r, d_{\text{in}}] = [8, 9216]$
  - B-head: latent $[8, 512] \to B\,[d_{\text{out}}, r] = [2304, 8]$
- Target modules: down_proj (MLP, хранит фактические знания).
- Обработка длинных документов: документ разбивается на чанки по 512 токенов, результирующие A-матрицы конкатенируются по рангу: $A = [A^{(1)}; A^{(2)}; \ldots; A^{(K)}]$, что увеличивает эффективный ранг до $r \times K$.

3.2.2 Прохождение обучающего примера

Документ c подаётся в замороженную базовую LLM, которая возвращает per-layer активации $Z \in \mathbb{R}^{L \times N \times D}$ ($L = 26$, N – число токенов, $D = 2304$). Для каждого слоя l общая гиперсеть h_ϕ:

1. получает активации предыдущего слоя Z_{l-1};
2. latent-запросы $Q_m \in \mathbb{R}^{r \times d_q}$ выполняют cross-attention к Z_{l-1}, формируя U_l;
3. линейные головки генерируют A_l и B_l.

Все 26 слоёв обрабатываются батчем, на выходе – тензоры $A[1, 26, 8, 9216]$, $B[1, 26, 8, 2304]$.

Функция потерь: дистилляция (KL-дивергенция)

$$\mathcal{L} = \mathbb{E}_{(c,D_c)} \mathbb{E}_{(x,y)\sim D_c} \mathrm{KL}\big(p_\theta(y|x,c) \parallel p_{\theta+H_\phi(c)}(y|x)\big),$$

где teacher – LLM с документом в контексте, student – LLM с LoRA-адаптером (без документа).

3.2.3 Обучающие данные

- Датасеты: fw_qa_v2, pwc, squad, ropes, drop.
- Режим self-distillation: teacher (gemma-2-2b-it с документом) генерирует ответы на автоматически составленные вопросы.

3.2.4 Параметры обучения

Обучение двухэтапное:

1. Стадия 1 (стабилизация): только 1 чанк на вход, 80 000 шагов.
2. Стадия 2 (chunking): случайное число чанков (1–8), шаги не уточнены. Наш чекпоинт соответствует 20 000 шагов (вероятно, конец стадии 1 или начало стадии 2).

Параметр	**Значение**
Learning rate	2e-5
Точность	bf16
Base model	google/gemma-2-2b-it
Параметров гиперсети	$\approx$ 315M
Параметров адаптера	2.4M (4.8 MB)
LoRA rank	8
LoRA alpha / scaling	45.25 / 5.657

Batch size в публичных источниках не указан.

3.3 Параметры конфигурации

Параметр	Значение
Base model	google/gemma-2-2b-it
ctx_encoder_type	per_layer_activations (нет отдельного энкодера)
aggregator_type	perceiver (Perceiver Resampler)
Num blocks агрегатора	9
latent_size	512
n_latent_queries	8
max_ctx_chunk_len	512 токенов (разбивка документа)
Target modules	down_proj
Rank (r)	8
Alpha / Scaling	45.25 / 5.657
Параметры гиперсети	315 237 792
Параметров адаптера	2 396 160
Обучающих шагов	20 000
Learning rate	2e-05
Train datasets	fw_qa_v2, pwc, squad, ropes, drop (self_gen от gemma-2-2b-it)

3.4 Запуск на Tesla T4

Результат: model.internalize(doc) и model.generate() работают корректно на T4. run_eval.py не запускается по той же причине, что и T2L - несовместимость vLLM с T4.

3.5 Анализ сгенерированных весов

Структура генерируемых тензоров: {"down_proj": {"A": [1, 26, 8, 9216], "B": [1, 26, 8, 2304]}}. Оба тензора создаются динамически для каждого документа - они не хранятся в state_dict модели.

Нормы и косинусное сходство матриц A и B для двух документов (doc1: Sakana AI wiki, doc2: станция Aurora):

Слой	**$\|A\|$ doc1**	**$\|A\|$ doc2**	**$\Delta\|A\|$**	**$\|B\|$ doc1**	**$\|B\|$ doc2**	**$\Delta\|B\|$**	**cos A**	**cos B**
0	5.7830	3.8410	1.9420	0.0205	0.0148	0.0057	0.581	0.594
1	4.9199	3.6213	1.2986	0.0262	0.0203	0.0058	0.639	0.605
2	5.2252	2.7313	2.4939	0.0358	0.0206	0.0152	0.518	0.383
5	5.6714	3.4310	2.2404	0.0384	0.0293	0.0091	0.561	0.339
8	5.3438	3.4353	1.9085	0.0517	0.0411	0.0105	0.474	0.407
10	5.0642	2.6054	2.4587	0.0647	0.0383	0.0264	0.301	0.220
12	4.4410	2.2514	2.1895	0.1320	0.0707	0.0614	0.170	0.159
15	6.4365	2.9231	3.5134	0.0843	0.0609	0.0234	0.538	0.177
19	7.1386	2.8404	4.2982	0.0745	0.0498	0.0247	0.511	0.146

Слой	$\|A\|$ doc1	$\|A\|$ doc2	$\Delta\|A\|$	$\|B\|$ doc1	$\|B\|$ doc2	$\Delta\|B\|$	cos A	cos B
20	6.1802	4.3787	1.8016	0.0665	0.0614	0.0051	0.647	0.206
25	5.3456	3.2543	2.0913	0.0386	0.0251	0.0135	0.510	0.350

Наблюдения:

- Нормы A существенно больше норм B (≈4–7 vs ≈0.02-0.13). A задаёт пространство проекции (более стабильное), B определяет документ-специфичное направление в этом пространстве.
- Глобальное косинусное сходство A = 0.56, B = 0.27 - матрица B в 2 раза более документ-специфична, чем A.
- Средние слои (10–19) имеют минимальный cosine B (≈0.15-0.22) - наибольшая специфика к содержимому именно там.

3.6 Демонстрация генерации

Вопрос / документ	Без адаптации (базовая модель)	С адаптацией (после internalize/generate_lora)
D2L: Документ - статья о SakanaAI	Sakana AI is a research lab focused on **developing and applying large language models (LLMs) for various applications, particularly in the realm of Japanese language understanding and generation.**	Sakana AI is a deep learning company based in Tokyo, Japan. Its main research directions are evolution and collective intelligence of AI.
D2L: Документ - описание станции Aurora (вымышленный)	The Aurora station, a research outpost in Antarctica, is primarily a research facility for the United States Antarctic Program (USAP). It houses a small team of researchers, typically around 1.	The Aurora station houses 14 researchers and is resupplied every 6 weeks.
D2L: Документ - техническое описание LoRA	LoRA: Efficient Fine-Tuning for Large Language Models. LoRA (Low-Rank Adaptation) is a technique for efficiently fine-tuning large language models (LLMs) by only updating a small subset of the mode	LoRA (Low-Rank Adaptation) is a parameter-efficient fine-tuning method that updates only a small portion of the weights in a model. Here's how LoRA computes weight updates: delta = b * A * (alp

Ключевые наблюдения:

- D2L корректно извлекает конкретные факты из документа: числа, названия, утверждения - включая вымышленные факты (станция Aurora), которые модель не могла знать из обучения.

- Оба метода работают без расширения контекста - знания закодированы в параметрах адаптера.
- С адаптацией ответ становится лаконичнее

4 Сравнение библиотек

4.1 Сводная таблица

Характеристика	Text-to-LoRA (T2L)	Doc-to-LoRA (D2L)
Назначение	Адаптация к типу задачи по её текстовому описанию	Усваивание знаний из произвольного документа
Тип входных данных	Короткий промпт (описание задачи)	Произвольный документ (статья, wiki, технический текст)
Контекстный энкодер	Отдельная модель gte-large-en-v1.5 (embedding 2048-dim, linear encoder)	Активации слоёв базовой LLM (per_layer_activations, нет отдельной модели)
Агрегатор	MLP / linear projection head (latent_size 512)	Perceiver Resampler (9 блоков, n_latent_queries=8, latent_size=512)
Target modules (LoRA)	q_proj, v_proj (механизм внимания)	down_proj (MLP - «фактические знания»)
LoRA rank (r)	8	8
LoRA alpha / scaling	16 / 2.0	45.25 / 5.657
LoRA параметров	1.6M, 3.2 MB	2.4M, 4.8 MB
Параметры гиперсети	130M	315M
Base model	google/gemma-2-2b-it	google/gemma-2-2b-it
Обучающих шагов	20 000 (эпох)	20 000 (шагов)
Learning rate	2.5e-5	2e-5
Обучающие данные	479 задач (FLAN, LoRA Zoo и др.)	fw_qa_v2, pwc, squad, ropes, drop (self_gen)
Генерация адаптера	1-2 сек (1 forward pass)	2-5 сек (forward pass + Perceiver)
API	generate_lora(task_description) → adapter	model.internalize(doc) → model.generate()

4.2 Сравнение режимов обучения T2L и D2L

В дополнение к архитектурным различиям, методы используют разные подходы к обучению гиперсети. Ниже приведена сводка из статей и конфигурационных файлов.

Характеристика	T2L (SFT)	D2L
Функция потерь	Cross-entropy (next-token prediction)	KL-дивергенция (дистилляция)
Вход гиперсети	эмбеддинг описания задачи [2048]	per-layer активации LLM при прогоне документа
Выход гиперсети	LoRA для q_proj, v_proj	LoRA для down_proj
Шаги обучения	20 000	80 000 (ст.1) + N (ст.2)
Learning rate	2.5e-5	2e-5
Параметры гиперсети	≈ 130M	≈ 315M
Обучающие данные	479 задач SNI	fw_qa_v2, pwc, squad, ropes, drop

4.3 Нормы весов: T2L vs D2L

Сравнение норм весов показывает принципиально разные паттерны специализации по слоям:

Слой	T2L math qA	T2L logic qA	T2L code qA	D2L sakana A norm	D2L aurora A norm	D2L cos A	Интерпретация
0	2.16	1.98	1.95	5.78	3.84	0.581	Ранние слои: низкая специализация
1	2.28	2.72	2.28	4.92	3.62	0.639	
5	2.11	2.01	2.04	5.67	3.43	0.561	
10	2.28	2.16	2.26	5.06	2.61	0.301	Средние слои: растёт расхождение
12	—	—	—	4.44	2.25	0.170	Мин. cosine D2L - макс. специфика
15	2.17	2.18	2.10	6.44	2.92	0.538	
19	—	—	—	7.14	2.84	0.511	Пик норм D2L
20	3.00	2.51	2.60	6.18	4.38	0.647	Поздние слои: высокая специализация T2L
21	3.22	2.38	2.76	6.44	3.01	0.598	Макс. расхождение T2L (Δ=0.84)
23	3.35	2.64	2.99	6.46	3.06	0.680	
25	2.79	2.59	2.49	5.35	3.25	0.510	Последний слой

Интерпретация различий:

- T2L: нормы монотонно нарастают к поздним слоям (0→25: 2.1→3.3). Специализация сосредоточена в верхних слоях - там, где формируются высокоуровневые, задача-специфичные паттерны.
- D2L: нормы A значительно выше (4-7) и немонотонны, с пиком в слоях 15–19. Это отражает то, что per_layer_activations кодируют документ на всех уровнях абстракции одновременно.
- Косинусное сходство D2L B (0.27 глобально) ниже, чем у T2L между разными задачами - D2L-адаптеры более документ-специфичны, чем T2L-адаптеры задача-специфичны.

4.4 Результаты оценки

Модель	ArcC	ArcE	BoolQ	GSM8K	HS	PIQA	WG	AVG
gemma-2-2b-it (base)	73.63	89.86	80.98	55.27	54.99	70.84	53.83	60.76
gemma + ICL	72.10	88.80	82.29	55.27	55.59	67.68	53.35	63.45
MT LoRA + ICL	74.06	88.80	81.01	56.41	60.09	68.17	58.01	65.07
T2L + ICL	74.12	90.03	82.27	56.61	62.87	73.61	57.51	66.09

T2L превосходит базовую модель на +5.3% avg и ICL-only на +2.6% avg по 10 бенчмаркам без какого-либо fine-tuning. Источник: таблица в README github.com/SakanaAI/text-to-lora

4.4.1 Doc-to-LoRA (из статьи https://arxiv.org/pdf/2602.15902)

Метрика (из pub.sakana.ai)	Значение
SQuAD: D2L vs full-context upper bound	83.5% от upper bound (без документа в контексте)
Long-context QA: D2L relative accuracy	85% от oracle (sub-second latency)
Oracle CD relative accuracy	90% (но 40 сек + >7 GB VRAM)
NIAH: точность до ≈40K токенов	Near-perfect (при training до 256 токенов)
Память: D2L vs KV-cache 128K doc	<50 MB vs >12 GB
VLM→LLM zero-shot (Imagenette 10-class)	75.03% accuracy
Eval на T4 (наш запуск)	Не выполнялся

D2L предоставляет только относительные метрики (% от full-context upper bound), а не абсолютные числа. Абсолютные значения воспроизводятся через scripts/main_exp/eval/*.sh, который на T4 не запускался.

5 Как библиотеки связаны

5.1 ОБЩАЯ ИДЕЯ (сходства)

Оба метода - это гиперсети, которые генерируют LoRA-адаптер
за один forward pass без градиентного обновления весов модели.
Общая архитектурная цепочка одинакова:
[входной сигнал] → [энкодер] → [агрегатор] → [HyperLoRA] → [LoRA-адаптер] → [LLM]
Общие зафиксированные параметры:
- Base model: google/gemma-2-2b-it (одна и та же)
- LoRA rank: r=8
- Обучающих шагов: 20 000
- Формат адаптера: совместим с PEFT
Общая концепция обучения:
- Оба обучаются через downstream loss (SFT-режим)
- Оба не требуют градиентов при inference
- Оба амортизируют стоимость адаптации: дорого один раз
обучить гиперсеть, дёшево генерировать адаптеры потом

5.2 Различия

ВХОДНОЙ СИГНАЛ:
T2L: строка текста (описание задачи, 100-500 символов)
D2L: произвольный документ (статья, wiki, любой длины)
КОНТЕКСТНЫЙ ЭНКОДЕР:
T2L: отдельная модель gte-large-en-v1.5
encode(text) → вектор [2048]
модель не является базовой LLM
D2L: сама базовая LLM (gemma-2-2b-it)
forward(doc_chunk) → активации всех 26 слоёв
каждый токен даёт 26 векторов по 2304 dim
документ разбивается на чанки по 512 токенов
АГРЕГАТОР:
T2L: MLP / linear projection head
вектор [2048] → latent [512]
простое линейное преобразование
D2L: Perceiver Resampler (9 блоков cross-attention)
активации [tokens × 26, 2304] → latent [8, 512]
8 обучаемых latent queries опрашивают документ
выход фиксированного размера при любой длине документа
TARGET MODULES (куда вставляется LoRA):
T2L: q_proj, v_proj - механизм self-attention

D2L: down_proj - MLP (финальная проекция)
ПАРАМЕТРЫ LORA:
T2L: alpha=16, scaling=2.0
D2L: alpha=45.25, scaling=5.657

5.3 Идеи объединения

ИДЕЯ 1: Единый входной интерфейс
Сейчас два разных API. Можно сделать один класс:

```
adapter = UniversalHyperNet.from_text(task_description)
adapter = UniversalHyperNet.from_doc(document)
adapter = UniversalHyperNet.from_both(doc, task_description)
```

Третий вариант - hybrid режим: документ задаёт знания,
описание задачи задаёт формат ответа.
ИДЕЯ 2: Двухэтапная адаптация
Не объединять в один адаптер, а применять последовательно:
Шаг 1: model.internalize(doc) # D2L адаптер - знания
Шаг 2: model.apply_task(task) # T2L адаптер - поведение
Шаг 3: model.generate(query)
Технически это композиция двух LoRA - уже поддерживается в PEFT
через add_weighted_adapter(). Минус: два отдельных forward pass
через гиперсети, два адаптера в памяти.